SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems
COURSE CODE: CSA0509

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency.* (BL3, BL4)

Activity Tool : Comparative Analysis (Low)
Assessment Tool Weightage: 10%

QUESTIONS		
Q.No	**Question**	**Bloom's Level**
1	Compare sequential file organization and heap file organization in terms of data insertion, deletion, and retrieval efficiency.	Bloom's Level: 4 – Analyze
2	Differentiate between clustered indexing and non-clustered indexing with suitable database examples.	Bloom's Level: 4 – Analyze
3	Compare primary indexing and secondary indexing based on storage requirements and search performance.	Bloom's Level: 4 – Analyze
4	Analyze the differences between static hashing and dynamic hashing in database systems.	Bloom's Level: 4 – Analyze
5	Compare B-Tree indexing and B+ Tree indexing for large-scale database applications.	Bloom's Level: 4 – Analyze
6	Differentiate linear hashing and extendible hashing in terms of bucket management and scalability.	Bloom's Level: 4 – Analyze
7	Compare dense indexing and sparse indexing with respect to memory usage and access speed.	Bloom's Level: 4 – Analyze
8	Analyze the advantages and limitations of indexed sequential file organization versus direct file organization.	Bloom's Level: 4 – Analyze

9	Compare single-level indexing and multi-level indexing for efficient database retrieval operations.	Bloom's Level: 4 – Analyze
10	Evaluate the performance of hashing techniques and indexing techniques for exact-match and range queries in databases.	Bloom's Level: 5 – Evaluate

---

Code of Conduct:

I, Judy Allen J (192520055) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
Judy Allen J

RUBRICS FOR CALCULATION:

Criteria	Weigh tage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Compari son Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidenc e	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation; lacks depth	No meaningful interpretation
Clarity of Presenta tion	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

1. Compare sequential file organization and heap file organization in terms of data insertion, deletion, and retrieval efficiency.

Answer:

Sequential file organization stores records in a specific order, usually according to a key field. Heap file organization stores records in no particular order and generally places new records wherever free space is available.

Aspect	Sequential File Organization	Heap File Organization
Record arrangement	Records are maintained in sorted/sequential order	Records are stored without any particular order
Insertion	Slower because the correct position must be found and records may need rearrangement	Very fast because the record can be inserted into available space
Deletion	May require marking the record deleted and later reorganizing the file	Relatively simple because the record can be removed directly
Sequential retrieval	Very efficient	Less efficient
Exact record search	Efficient when records are ordered by the search key	Usually requires a complete file scan if no index exists
Range queries	Highly efficient	Poor because records are unordered
Best suited for	Applications involving frequent sequential processing	Applications involving frequent insertions

Example:

A payroll system that processes employees in Employee_ID order benefits from sequential organization. A temporary transaction table receiving a large number of new records benefits from heap organization.

Conclusion:

Sequential organization is better when ordered and range-based retrieval is important, whereas heap organization is preferable when fast insertion is the main requirement.

2. Differentiate between clustered indexing and non-clustered indexing with suitable database examples.

Answer:

Clustered and non-clustered indexes improve database retrieval, but they differ mainly in how they relate to the physical storage of table records.

Aspect	Clustered Index	Non-Clustered Index
Data arrangement	Determines the physical/logical ordering of table records	Maintains a separate index structure
Number per table	Generally one clustered organization can exist	Multiple non-clustered indexes can exist
Leaf nodes	Contain the actual table data or clustered records	Contain index keys and references to records
Retrieval	Very efficient for ordered and range searches	Efficient for searches using indexed columns
Storage	Requires less additional pointer structure than multiple secondary indexes	Requires extra storage for index entries
Range queries	Very efficient	Efficient, but may require additional record lookups
Insert/update cost	Can be higher because record ordering may need adjustment	Index entries must be updated separately

Database Example:

Consider an Employee table containing:

Employee(Emp_ID, Name, Department, Salary)

If the table is organized using Emp_ID, records can be stored according to the clustered key. A separate index on Department can act as a non-clustered or secondary index to quickly locate employees belonging to a particular department.

For example:

CREATE INDEX idx_department
ON Employee(Department);

This index allows the DBMS to search employees based on Department without scanning every row.

Conclusion:

Clustered indexing is especially useful for range and ordered retrieval, while non-clustered indexing provides flexibility by allowing multiple search paths over the same table.

3. Compare primary indexing and secondary indexing based on storage requirements and search performance.

Answer:

Primary indexing is created on the primary ordering key of a file, while secondary indexing provides an additional access path using a non-ordering attribute.

Aspect	Primary Index	Secondary Index
Index field	Usually based on primary key/order key	Based on non-ordering fields
Data organization	Records are generally organized according to the indexed key	Physical record order is independent of the index
Density	Can often be sparse	Usually dense
Storage requirement	Lower	Higher
Search performance	Very efficient for primary-key searches	Efficient for searches on alternate attributes
Number of indexes	Normally associated with the main ordering key	Multiple secondary indexes may exist
Maintenance	Relatively lower overhead	Higher overhead when inserts, deletes or updates occur

Example:

Consider:

Student(Student_ID, Name, Department, Email)

A primary index can be based on Student_ID. A secondary index may be created on Department to retrieve students belonging to a particular department.

```
CREATE INDEX idx_student_department
ON Student(Department);
```

Analysis:

Primary indexing consumes less additional storage because the data itself is organized using the primary key. Secondary indexing consumes more space because an additional index structure must maintain references to records.

Conclusion:

Primary indexes provide excellent performance for primary-key retrieval with lower storage overhead, while secondary indexes use additional storage but provide more flexible search options.

4. Analyze the differences between static hashing and dynamic hashing in database systems.

Answer:

Hashing uses a hash function to determine the storage location of records. Static and dynamic hashing differ mainly in how they respond to changes in database size.

Aspect	Static Hashing	Dynamic Hashing
Number of buckets	Fixed	Can increase or decrease
Hash structure	Remains constant	Changes dynamically
Database growth	Less suitable	Highly suitable
Overflow handling	Overflow buckets or chaining may be required	Buckets can be split when needed
Performance	Good when file size is stable	Maintains good performance as data grows
Storage overhead	Lower	Slightly higher because extra directory/control information may be required
Complexity	Simple	More complex
Scalability	Limited	High

Example:

Suppose a student
initially contains 1,000 records. In static hashing, a fixed number of buckets is allocated. If the database grows to 50,000 records, many bucket overflows may occur.
With dynamic hashing, additional buckets can be created as the number of records increases, reducing overflow and maintaining faster retrieval.

Analysis:

Static hashing is suitable for databases whose size does not change significantly. Dynamic hashing is more appropriate for modern applications where records are frequently inserted and deleted.

Conclusion:

Static hashing offers simplicity and low overhead, whereas dynamic hashing offers superior scalability and more consistent performance for growing databases.

5. Compare B-Tree indexing and B+ Tree indexing for large-scale database applications.

Answer:

B-Tree and B+ Tree are balanced tree structures used for database indexing. Both maintain logarithmic search performance, but their internal organization differs.

Aspect	B-Tree	B+ Tree
Data storage	Keys and record references may be stored in internal and leaf nodes	Record references are primarily stored at leaf nodes
Internal nodes	Store keys and possibly record references	Mainly store search keys and child pointers
Leaf nodes	Not necessarily linked sequentially	Usually linked sequentially
Exact search	Efficient	Efficient
Range search	Good	Very efficient
Sequential access	Less convenient	Excellent
Fan-out	Lower because internal nodes store more information	Higher because internal nodes store mostly keys and pointers
Tree height	May become slightly taller for equivalent layouts	Often remains relatively shallow due to higher fan-out
Database usage	Less commonly preferred for disk-based DB indexing	Widely used in database systems

Example:

Consider millions of banking transaction records arranged by Transaction_ID. A B+ Tree allows the DBMS to locate the starting record quickly and then traverse linked leaf nodes for a range such as Transaction_ID 10000 to 20000.

Analysis:

For large databases, minimizing disk access is important. B+ Trees have high fan-out, allowing more keys per internal node and reducing the number of disk pages that may need to be accessed.

Conclusion:

Both structures provide efficient searching, but B+ Tree indexing is generally more suitable for large-scale database systems, especially where range queries and sequential access are common.

6. Differentiate linear hashing and extendible hashing in terms of bucket management and scalability.

Answer:

Linear hashing and extendible hashing are dynamic hashing techniques designed to handle database growth without completely rebuilding the hash structure.

Aspect	Linear Hashing	Extendible Hashing
Directory	Does not require a large directory	Uses a directory
Bucket splitting	Buckets are split gradually according to a predefined sequence	Selected buckets are split based on directory depth
Growth	Incremental	Dynamic using directory expansion
Memory requirement	Lower	Higher because a directory must be maintained
Bucket management	Uses split pointers and hash levels	Uses global depth and local depth
Scalability	Good	Very good
Overflow	May temporarily use overflow buckets	Usually controls overflow efficiently through bucket splitting
Complexity	Relatively simple	More complex

Example:

In linear hashing, when records increase, buckets are split one after another gradually instead of immediately doubling the entire structure.

In extendible hashing, directory entries point to buckets. When a bucket becomes full, the bucket may split, and the directory may double if required.

Analysis:

Linear hashing minimizes directory overhead and provides smooth growth. Extendible hashing offers more flexible bucket selection and can provide very efficient lookup but requires additional memory for the directory.

Conclusion:

Linear hashing is preferred when lower memory overhead is important, while extendible hashing is suitable for rapidly growing systems requiring efficient dynamic bucket management.

7. Compare dense indexing and sparse indexing with respect to memory usage and access speed.

Answer:

Dense and sparse indexes differ according to the number of index entries maintained for database records.

Aspect	Dense Index	Sparse Index
Index entries	Contains an entry for every search-key value or record	Contains entries for selected records/blocks
Memory usage	Higher	Lower
Storage requirement	High	Low
Search speed	Faster	Slightly slower
Record location	Can directly identify a record or matching record set	Locates a nearby block and then searches within it
Maintenance	More costly	Less costly
Suitable for	Fast exact searching	Large ordered files where memory efficiency matters

Example:

Suppose a database has 10,000 student records.

A dense index may contain index entries corresponding to all 10,000 records or search-key occurrences.

A sparse index might store only one entry for each disk block. If each block contains 100 records, approximately 100 index entries may be enough.

Analysis:

Dense indexing trades increased storage for faster retrieval. Sparse indexing reduces memory requirements but requires an additional search after the correct block is identified.

Conclusion:

Dense indexing provides higher access speed, while sparse indexing provides better memory efficiency. The choice depends on whether retrieval speed or storage efficiency is more important.

8. Analyze the advantages and limitations of indexed sequential file organization versus direct file organization.

Answer:

Indexed sequential file organization combines sequential record storage with an index, whereas direct file organization generally uses a calculated address, often through hashing, to access records directly.

Aspect	Indexed Sequential Organization	Direct File Organization
Record organization	Records are maintained sequentially with an index	Record location is calculated directly
Exact-match search	Efficient	Very efficient
Range query	Excellent	Poor
Sequential processing	Excellent	Not naturally suited
Insertion	May require overflow areas/reorganization	Generally efficient
Index requirement	Requires index storage	Usually uses hashing/address calculation
Maintenance	Higher because both records and indexes must be maintained	Depends on collision and overflow handling
Best application	Systems requiring both sequential and random access	Exact-key retrieval systems

Example:

A bank may use indexed sequential organization for customer records because records may need to be processed both individually and sequentially.

A direct organization approach can be useful where an account number is transformed into a storage location using a hash function.

Advantages of Indexed Sequential Organization:

It supports both sequential access and direct indexed access, performs well for range queries, and allows ordered processing.

Limitations:

It requires additional index storage, overflow areas may develop after repeated insertions, and periodic reorganization may be necessary.

Advantages of Direct Organization:

Exact-match retrieval is extremely fast and insertion can be efficient.

Limitations:

It performs poorly for range queries and may suffer from hash collisions and bucket overflow.

Conclusion:

Indexed sequential organization is more versatile, while direct organization is preferable when exact-match retrieval dominates the workload.

9. Compare single-level indexing and multi-level indexing for efficient database retrieval operations.

Answer:

Single-level indexing uses one index structure to locate records, while multi-level indexing creates additional indexes over lower-level indexes.

Aspect	Single-Level Indexing	Multi-Level Indexing
Number of index levels	One	Two or more
Search process	Searches one index before accessing data	Traverses multiple index levels
Suitable database size	Small or medium databases	Very large databases
Disk accesses	Can increase as the index grows	Reduced due to hierarchical searching
Storage	Lower	Higher
Search efficiency	Good	Very high for large databases
Scalability	Limited	High
Structure	Simple	Hierarchical

Example:

Suppose a database contains 1,000,000 employee records. A single-level index itself may become very large and require several disk accesses.

With multi-level indexing, a smaller upper-level index points to blocks of the lower-level index. Therefore, only a small number of index blocks need to be accessed before reaching the desired record.

Analysis:

Multi-level indexing applies the same idea recursively until the highest-level index becomes small enough to search efficiently.

B-Tree and B+ Tree structures implement this concept efficiently by maintaining balanced hierarchical index levels.

Conclusion:

Single-level indexing is simpler and suitable for smaller datasets. Multi-level indexing offers better scalability and efficient retrieval for large-scale databases.

10. Evaluate the performance of hashing techniques and indexing techniques for exact-match and range queries in databases.

Answer:

Hashing and indexing are both used to improve database search performance, but their efficiency depends heavily on the type of query being performed.

Aspect	Hashing	Indexing
Exact-match query	Extremely efficient	Very efficient
Range query	Poor	Excellent, especially with B+ Trees
Record ordering	Does not maintain sorted order	Can maintain ordered keys
Search method	Computes bucket/address using hash function	Traverses an index structure
Average exact search	Can approach constant-time bucket lookup under good conditions	Usually logarithmic for tree indexes
Sequential retrieval	Poor	Good
Collision problem	Possible	Not applicable in the same way
Dynamic growth	Requires techniques such as linear/extendible hashing	Balanced indexes adapt well
Best application	Equality searches	Mixed equality and range searches

Exact-Match Query

Consider:

```
SELECT *
FROM Employee
WHERE Employee_ID = 105;
```

For an exact-match query, hashing can be highly efficient because the hash function calculates the bucket in which the required record should be located.

An index on Employee_ID also performs efficiently, but a tree index generally requires traversal through multiple nodes.

Therefore, for pure exact-match queries:

Hashing → generally excellent

Range Query

Consider:

```
SELECT *
FROM Employee
WHERE Salary BETWEEN 50000 AND 80000;
```

Hashing is inefficient for this query because hash values do not preserve the natural ordering of salaries.

An ordered index such as a B+ Tree can locate the first qualifying salary and then continue through adjacent leaf entries.
Therefore, for range queries:
B+ Tree / ordered indexing → generally superior

Performance Evaluation
Hashing is ideal for applications where most searches use equality conditions, such as:
WHERE Account_No = 12345
Indexing is more versatile because it supports exact searches, range searches, ordering and sequential retrieval.
For example:
CREATE INDEX idx_salary
ON Employee(Salary);
This index can improve searches involving salary values, particularly range-based retrieval.

Conclusion
The appropriate technique depends on workload. Hashing is generally the better choice for exact-match retrieval when a suitable hash implementation is available, while ordered indexing, especially B+ Tree indexing, is better for range queries and mixed workloads. Thus, modern database systems often use indexing for general-purpose retrieval and hashing where equality-based access is dominant.